Declan Sheehan
Dr. Anderson
COSC320-001
25 September, 2019

Lab 4 Read Me

Approach Summary:

Generally, I followed pseudo-code given, and my background knowledge on heaps, heapify, and queues. First, I created the class for the heap (HeapQ), then I worked on inserting nodes (because you cannot test a queue without data). Next was class routines, such as Increasekey, Maxheapify, Peek, Expandheap, and Print. I knew that the main had to create and instance of the class (HeapQ) to then manipulate the attached dynamic array of heap objects (HeapObj). Since the heap we construct uses an array, the abstract aspect of the functions was to fill the array so that the heap or array follows heap properties.

Time Complexity:

The time complexity for `Increasekey` is O(log(n)). This is because the algorithm traverses down the heap to place the correct node in its position. The heap from node to root is O(log(n)).

The time complexity for `Insert` is also O(log(n)), since it calls Increasekey; 2 + O(log(n)) = O(log(n)).

The time complexity for `Maxheapify` is O(log(n)) as it traverses through the heap, where each child's subtree has the size of <= 2n/3.

The time complexity for `Extractheap` is also O(log(n)) since it is again a constant plus O(log(n)) from Maxheapify.

Best case for all of these should be *theta*(1), assuming we have a small array.

Note: n = array size/number of objects in heap.

Absolute Timing:

When timing each function, it took a very very small amount of time per operation. I was only able to take a small sample of the time:

N = 10------ 3.482e-06s
N = 100----- 1.6201e-05s
N = 1000---- 0.000244835s

Applications:

In the delivery business, efficiency and proper export timing has a tremendous effect on service and revenue. Say for instance; two identical packages are bought and ordered at the same time, but by different people that happen to live near one another. To add to it, one of the buyers have premium shipping, while the other does not. Ordering systems and servers would need to communicate which package to send out at what time based off of distance, membership status, package type, and more. A priority queue has those capabilities. Higher prioritized shipment orders are placed at the top, while lower ones are below. From what data structures that I personally know, this type seems the most practical, and helpful, with respect to operations. For instance, a large database-structure would not be able to calculate which package to prioritize, but could hold the package ordering details to then offload to a priority-queue.

Over the course of the Cold War, the United States has accumulated thousands of nuclear warheads. Unfortunately - this along with the use of fission reactors cause a lot of waste to be generated. Places around the U.S. are home to sources of nuclear waste, and outdated warheads that have a potent potential. A priority queue could manage a list of these sites, and prioritize their clean-up/maintenance based on the danger the site hosts. Maybe this will encourage the U.S. to act on it.

Code Improvements:

For the most part, the functions work. The exception being Expandheap when using strings as the class `T`. The output of the Print function could always be better, but it is fair. The Expandheap also segfaults when using strings.